Integrity, Signatures, and PKI

IE3122: Network Security | Week 12

Hashes, MACs, digital signatures, certificates, and how TLS uses them all

Today's Agenda

- Hash functions: what they guarantee, how they fail, and where they are used
- MACs and HMAC: adding a secret key to a hash
- Digital signatures: integrity, authentication, and non-repudiation together
- Digital certificates: binding an identity to a public key
- PKI: certificate authorities, chains of trust, revocation, and what happens when trust fails
- Putting it together: a full TLS handshake, and a short look at PKI on Cisco devices

Learning Outcomes for Today

LO3

– Analyse security requirements and recommend appropriate technologies.

LO4

– Evaluate different security technologies, compare them, and identify appropriate use cases.

LO5

– Configure security technologies to satisfy specific requirements (introduced at the end of this deck).

Hash Functions

A fingerprint for data

What a Hash Function Is

Property	Meaning	What Goes Wrong Without It
Fixed-size output	Any input, from one byte to a whole disk image, produces a digest of the same length	Digests could not be compared or stored uniformly
Deterministic	The same input always gives the same digest	Two parties could never agree that data is unchanged
One-way (preimage resistant)	Given a digest, it is infeasible to find any input that produces it	Stored password digests could be reversed
Second-preimage resistant	Given one input, it is infeasible to find a different input with the same digest	An attacker could swap a document for a forgery
Collision resistant	It is infeasible to find any two inputs with the same digest	Signed documents could be replaced by a colliding twin

The Avalanche Effect: A Worked Example

- Change one character of the message and the digest changes completely. Here are two messages that differ in a single digit, hashed with SHA-256 (first 32 of 64 hex characters shown):

Input	SHA-256 Digest (first 32 hex characters)
Pay Alice 100 dollars	2a31bcf4ca7789bc833e8aedb1e97f1b
Pay Alice 900 dollars	995a973968487cb07d6ebd53fb8379d0

- The two full 256-bit digests differ in 133 of 256 bits, close to the expected half. There is no visible relationship between the input change and the output change.
- This is why a digest works as a tamper alarm: an attacker cannot make a small, targeted edit and get a small, predictable digest change. It is the same avalanche idea met in Deck A, where one flipped input bit changes about half of a block cipher's output.

Hash Algorithms: MD5, SHA-1, SHA-2, SHA-3

Algorithm	Digest Size	Status	Notes
MD5	128 bits	Broken, do not use for security	Practical collisions since 2004. Collisions were used to forge a CA certificate in 2008.
SHA-1	160 bits	Deprecated	The first practical collision (SHAttered) was demonstrated in 2017. Browsers and CAs no longer accept it.
SHA-2 (SHA-256, SHA-512)	256 or 512 bits	Current standard	The default choice today. Used in certificates, TLS, IPsec, and software signing.
SHA-3	224 to 512 bits	Current, alternative	Standardised in 2015. A different internal design (Keccak), so a weakness in SHA-2 would not automatically affect it.

Recall the IPsec framework table in Week 9: this is the integrity row, where MD5 and SHA-1 have been replaced by SHA-2.

Attacks on Hashes

- Collision attacks: find two different inputs with the same digest. The birthday paradox means this needs only about 2^(n/2) work for an n-bit hash, so a 128-bit hash like MD5 offers just 64 bits of collision resistance, and SHA-256 offers 128. This is why digest length matters.
- Brute force and dictionary attacks: for short or predictable inputs such as passwords, the attacker simply hashes guesses until one matches. Hashing is fast, which helps the attacker here.
- Rainbow tables: precomputed tables of digests for common passwords, so an unsalted password digest is looked up rather than computed.
- Defences for passwords: add a unique random salt to each password before hashing so precomputed tables are useless, and use a deliberately slow password hashing function (bcrypt, scrypt, Argon2, or PBKDF2) instead of a plain fast hash.

Hash Use Cases

- File and download integrity: a software vendor publishes the digest of a download. You hash your copy and compare. Caution: the digest must reach you over a trusted channel, because an attacker who can replace the file can also replace a digest published beside it.
- Password storage: systems store a salted, slow hash of the password, never the password. A stolen database then does not directly reveal the passwords. Recall Week 10: this protects the local AAA user database.
- Digital signatures: signing a short digest is far cheaper than signing a whole message. This is the use that leads into the signature section.
- Forensics and evidence: a digest recorded when evidence is collected proves later that the evidence has not changed.

Hash Functions: Strengths, Weaknesses, Use Cases

Strengths

- Very fast, and the digest is small regardless of input size.
- No key to manage or distribute.
- Detects any accidental or deliberate change to data (with a secure SHA-2 or SHA-3 hash).
- A building block for MACs, signatures, and password storage.

Weaknesses

- No secret: anyone can compute a hash, so an attacker who alters a message can simply recompute the digest.
- Provides integrity only when the digest itself is protected or delivered securely.
- Fast hashing makes password guessing fast unless a slow password hash is used.
- Older algorithms (MD5, SHA-1) are broken, and old systems still use them.

Use cases: download verification, salted password storage, the digest step inside digital signatures and certificates, forensic evidence, and detecting file tampering.

MACs and HMAC

Adding a secret key to the digest

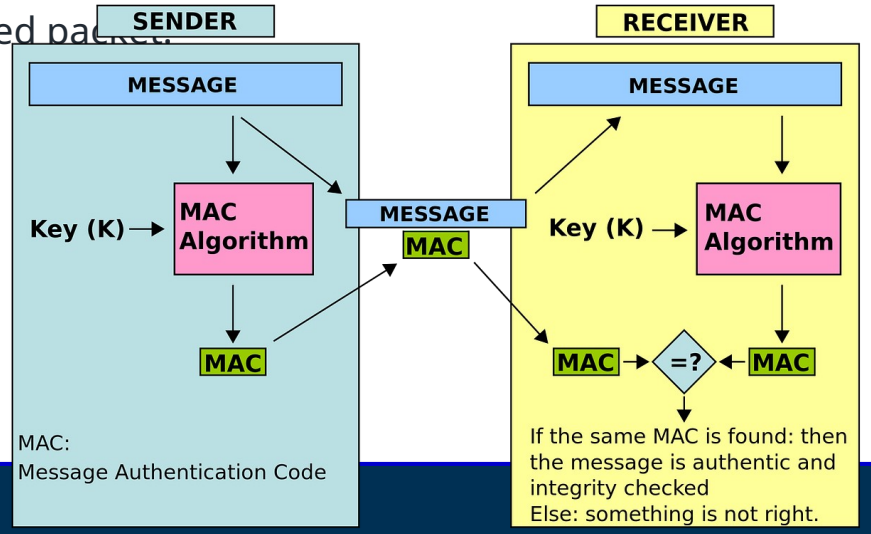
Why a Hash Alone Is Not Enough



- Mallory intercepts the message, edits it, and recomputes the digest. Bob checks the digest, finds it matches the message, and accepts the forgery. The hash proved the message was consistent, not that Alice sent it.
- The missing ingredient is a secret that Mallory does not have. Mixing a shared key into the calculation produces a Message Authentication Code (MAC): only someone who holds the key can produce a valid one.

MAC Concept and HMAC

- A MAC takes a message and a shared secret key and produces a short tag. The sender attaches the tag; the receiver recomputes it with the same key and compares. A match proves both integrity and that the sender held the key.
- HMAC is the standard way to build a MAC from a hash function. It hashes the message twice, mixing the key in at both stages: HMAC(K, m) = H((K xor opad) || H((K xor ipad) || m)). The double construction avoids weaknesses of naively hashing key and message together.
- Common forms: HMAC-SHA256 and HMAC-SHA512. The hash inside may even be one with known collision weaknesses, because HMAC is far more robust than the bare hash, though new designs should still use SHA-2.
- Callback: the integrity row of the IPsec framework table in Week 9 is HMAC. IPsec and TLS both attach a MAC (or use an authenticated mode such as GCM) to every protected packet.



MACs: Strengths, Weaknesses, Use Cases

Strengths

- Provides integrity and origin authentication among key holders.
- Fast: a hash and a few extra operations, with no asymmetric mathematics.
- Simple and well understood, with long-standing standards (RFC 2104).
- An attacker without the key cannot forge a valid tag.

Weaknesses

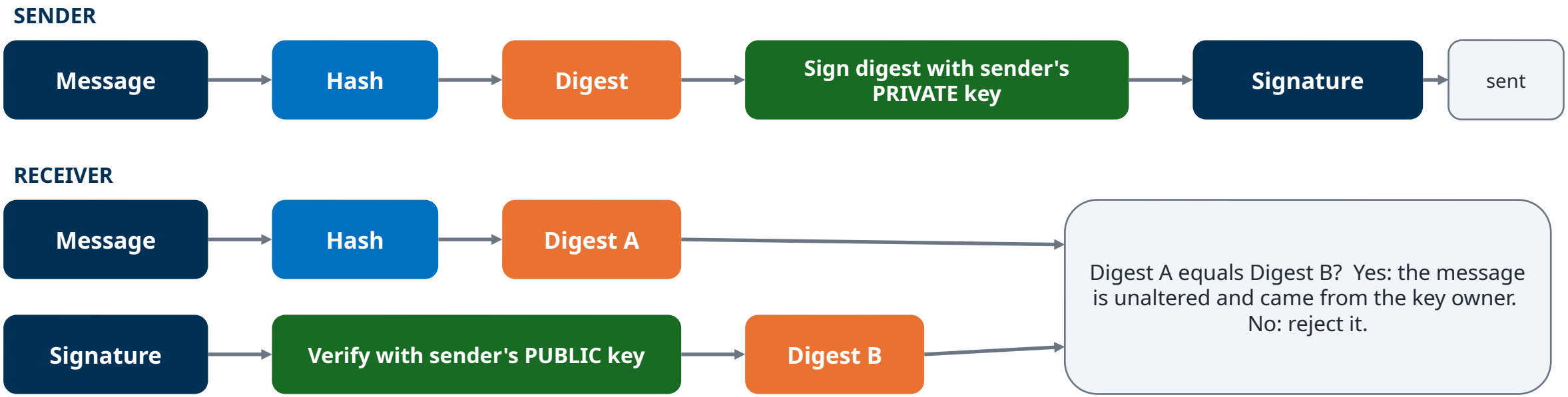
- The key distribution problem returns: both ends need the same secret in advance.
- No non-repudiation: both parties hold the key, so either could have produced any tag.
- A leaked key lets an attacker forge messages in both directions.
- Does not stop replay of a captured valid message unless a sequence number or timestamp is included.

Use cases: IPsec and TLS packet integrity, API request signing between servers, session tokens (such as JWT HS256), and routing protocol authentication (recall OSPF and EIGRP MD5 and SHA authentication in Week 5).

Digital Signatures

Proving who signed, and that nothing changed

How Signing and Verifying Work



- Sign the hash, not the whole message: asymmetric operations are slow, so the sender signs the short digest. Conceptually the digest is transformed with the private key; real schemes add padding or use signature-specific mathematics.
- Anyone with the public key can verify, but only the private key holder can sign.

What a Signature Guarantees

Guarantee	How the Signature Provides It
Integrity	Any change to the message changes its digest, so verification fails.
Authentication	Only the holder of the private key could have produced a signature that verifies with the matching public key.
Non-repudiation	Only the signer has the private key, so the signer cannot credibly deny signing. A third party can verify this without any shared secret.

- These guarantees hold only if two assumptions hold: the private key has stayed private, and the public key really belongs to the claimed signer. The second assumption is the subject of the certificate section.

Signature Algorithms: RSA, DSA, ECDSA, EdDSA

Algorithm	Based On	Typical Size	Status	Notes
RSA	Integer factoring	2048 to 3072 bit keys; signature as long as the key	Widely used	The most common in certificates. Use the PSS padding scheme for new designs.
DSA	Discrete logarithm	2048 bit keys	Legacy	No longer approved for creating new signatures in recent NIST standards. You may still meet it in old systems.
ECDSA	Elliptic curve	256 bit keys; signatures about 64 bytes	Widely used	Small and fast. Needs a fresh random value for every signature: reusing one leaks the private key.
EdDSA (Ed25519)	Elliptic curve	256 bit keys; signatures 64 bytes	Modern	Deterministic, so it avoids the random-value pitfall. Used in SSH and increasingly in TLS.

Why Non-Repudiation Needs Asymmetric Keys

MAC (shared key)

- Alice and Bob both hold the key, so either could have produced any given tag.
- If Alice later claims she never sent a message, Bob cannot prove otherwise: he could have made the tag himself.
- Good for protecting a conversation between two trusting parties. Useless as evidence for a third party.

Digital signature (key pair)

- Only Alice holds the private key, so only Alice could have signed.
- Anyone, including a judge or auditor, can verify with her public key. Bob cannot forge her signature.
- This is what makes contracts, software updates, and audit records provable.

Callback: Week 2 defined non-repudiation as a goal, and Week 10's command accounting recorded who ran what. A log is only strong evidence if it cannot be altered or denied afterwards, so high-assurance audit records are digitally signed.

Digital Signatures: Strengths, Weaknesses, Use Cases

Strengths

- Provides integrity, authentication, and non-repudiation together.
- Verifiable by anyone with the public key, with no shared secret.
- Scales: each signer needs only one key pair.
- Underpins certificates, secure updates, and code signing.

Weaknesses

- Slower than a MAC, and signatures are larger.
- Everything depends on the private key staying private: a stolen key lets an attacker sign as the owner.
- A signature only proves possession of a key. It does not prove who that key belongs to.
- Current algorithms would be broken by a large quantum computer.

Use cases: certificates, software and firmware update signing, code signing, signed email (S/MIME), contract and document signing, DNSSEC and RPKI (recall Week 5), and the handshake in TLS and SSH.

Hash vs MAC vs Digital Signature

Aspect	Hash	MAC / HMAC	Digital Signature
Key used	None	One shared secret key	Private key to sign, public key to verify
Integrity	Only if the digest is delivered securely	Yes	Yes
Authentication	No	Yes, among key holders	Yes, of the signer
Non-repudiation	No	No	Yes
Relative speed	Fastest	Fast	Slowest
Typical use	File checks, password storage	Packet integrity in IPsec and TLS	Certificates, software signing, contracts

The Hook: But Whose Public Key Is This?

- Everything so far assumes you hold the genuine public key of the other party. A signature verifies against a public key, but nothing in the mathematics says whose key that is.
- Suppose Mallory sits between you and a bank. When your browser asks for the bank's public key, Mallory supplies her own. Her signatures verify perfectly against her key. Her key exchange with you is just as valid. The mathematics works flawlessly, and you are talking to the attacker.
- Diffie-Hellman from Deck A has the same gap: it agrees a key, but not with whom.
- We need a trusted third party to vouch for the link between an identity and a key. That vouching document is a digital certificate.

Digital Certificates

Binding an identity to a public key

The Man-in-the-Middle Problem



- Encryption and signatures cannot detect this on their own. The client has no way to tell a genuine key from Mallory's.
- The fix is an independent, trusted party that has checked the server's identity and then signs a statement binding that identity to that key. The client trusts the signer, so it can trust the key. That statement is a certificate.

What a Certificate Is: X.509

- A digital certificate is a signed data structure that binds a subject (a person, a website, a device) to a public key. The standard format is X.509 (RFC 5280).
- It is signed by a Certificate Authority (CA) using the CA's private key. Anyone who trusts the CA and holds its public key can verify the signature and so trust the binding.
- Analogy: a passport. A government you trust checked who you are, bound your name to your photograph, and added a hard-to-forge seal. A border officer trusts the seal, not you. A certificate is the same with a name, a public key, and a digital signature.
- A certificate contains no secrets. It is public, and sent to anyone who asks. The private key that matches it never leaves its owner.

Certificate Anatomy

Field	What It Contains
Version and serial number	X.509 version (normally v3) and a number unique to this certificate within the issuing CA
Signature algorithm	How the CA signed it, for example SHA-256 with RSA or ECDSA
Issuer	The CA that issued and signed the certificate
Validity	The Not Before and Not After dates. Outside this window the certificate is invalid
Subject	The identity being certified, such as the website's domain name
Subject public key	The public key being bound to that identity, and its algorithm
Extensions	Subject Alternative Name (all valid hostnames), Key Usage, Basic Constraints (is this a CA?), and where to check revocation (CRL or OCSP)
CA signature	The issuer's digital signature over all of the above

How a Certificate Is Verified

- Step 1, dates: is today inside the Not Before to Not After window?
- Step 2, name: does the Subject or Subject Alternative Name match the host you meant to reach? A valid certificate for another site must fail.
- Step 3, signature: verify the CA's signature using the issuer's public key. This proves the contents have not been altered since issuance.
- Step 4, chain: is the issuer itself trusted? If it is an intermediate, repeat steps 1 to 3 up the chain until a root the client already trusts is reached.
- Step 5, revocation: has the certificate been revoked before it expired? (CRL or OCSP, covered in the PKI section.)
- Step 6, possession: the server proves it holds the matching private key by signing something during the handshake. Without this, anyone could replay a copied certificate.

Certificate Types: DV, OV, EV, Self-Signed

Type	What the CA Verifies	Typical Use	Trade-off
Domain Validated (DV)	Only that the applicant controls the domain name	Most websites, automated issuance	Fast and cheap, but says nothing about who is behind the site
Organisation Validated (OV)	Domain control plus the organisation's legal existence	Business and institutional sites	More assurance, slower issuance
Extended Validation (EV)	A more rigorous check of the organisation's identity	Banks and high-risk sites	Highest vetting, though browsers now display it far less prominently
Self-signed	Nothing: the subject signs its own certificate	Labs, internal tests, some devices	No third party vouches for it, so browsers warn and users must trust it manually

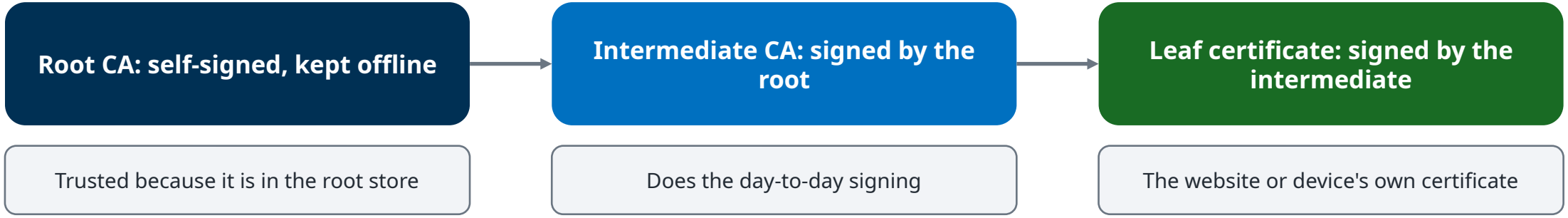
Public Key Infrastructure

The system that makes certificates trustworthy at scale

PKI Components

Component	Role
Certificate Authority (CA)	Issues, signs, and revokes certificates. Its private key is the root of trust and must be guarded extremely carefully.
Registration Authority (RA)	Verifies the applicant's identity on the CA's behalf, so that the CA's signing key stays isolated.
End entity (subscriber)	The person, server, or device that requests and uses a certificate.
Repository	A public directory where certificates and revocation information are published.
Relying party	Whoever receives a certificate and decides whether to trust it, such as a browser or a router.

Chain of Trust: Root, Intermediate, Leaf



- Why an intermediate: the root's private key is so valuable that it is kept offline and used rarely. An intermediate does the daily signing, so if it is compromised, only it needs to be revoked and replaced, not the root.
- A server normally sends its leaf and intermediate together. The client already holds the root, and verifies each link in turn, exactly as in verification step 4.

Root Stores: Trust Is a Policy Decision

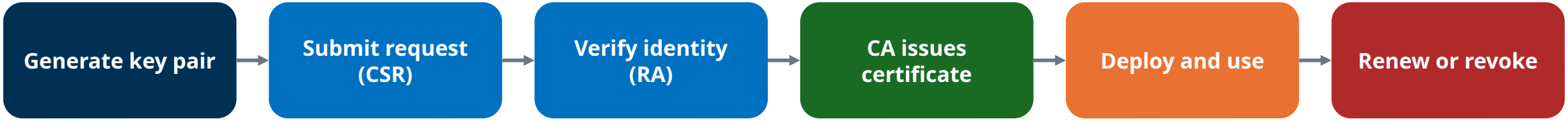
- A root certificate is trusted simply because it is already installed in the relying party's root store. Operating systems and browsers ship with a curated list of a few hundred roots, and their vendors decide who is on it.
- Trust is therefore a policy decision, not a mathematical fact. A CA earns a place by passing audits, and can be removed if it misbehaves.
- Organisations can add their own internal root through device management, so internal servers and devices are trusted without a public CA. Whoever controls that root can issue certificates the whole organisation will trust.
- Callback to Week 5: RPKI used the same idea of a chain of trust from a trust anchor, with regional internet registries at the top, to authenticate who owns an IP prefix. The structure is similar; the trust anchors are different.

Revocation: CRL vs OCSP

Aspect	CRL (Certificate Revocation List)	OCSP (Online Certificate Status Protocol)
How it works	The CA publishes a signed list of revoked serial numbers; clients download it	The client asks a CA responder about one certificate and gets a signed good, revoked, or unknown reply
Freshness	Only as current as the last published list	Near real time
Size and load	Lists can grow large	One small query per certificate, but heavy load on the responder
Privacy	No per-site information leaked	The responder learns which sites the client visits (reduced by OCSP stapling)
If the service is down	Cached copy may still work	Clients often fail open, which weakens the check

OCSP stapling lets the server attach a recent signed OCSP reply, so the client need not ask. The industry is also shifting back towards CRLs for scale and privacy reasons.

Certificate Lifecycle



- The applicant generates the key pair itself, so the private key never travels. The Certificate Signing Request (CSR) carries only the public key and identity details.
- Certificates expire on purpose. Short lifetimes limit the damage of a stolen key, which is why public web certificates are now valid for months rather than years and are renewed automatically.
- Revocation is for emergencies: a private key is stolen, a certificate was issued wrongly, or the subject no longer exists. The CA adds it to its CRL and OCSP responses.

Case Study: DigiNotar, 2011

- What happened: attackers broke into DigiNotar, a Dutch certificate authority, and issued fraudulent but fully valid certificates, including one for google.com. The signatures were genuine, because they were made with DigiNotar's real keys.
- Impact: the certificates were used to intercept the Gmail traffic of many users, mostly in Iran. Every browser trusted them, because DigiNotar was in every root store.
- Response: browser and operating system vendors removed DigiNotar from their root stores. Its certificates became untrusted overnight, and the company went bankrupt soon after.
- Lessons: any one trusted CA can issue a certificate for any domain, so the whole system is only as strong as its weakest CA. This led to Certificate Transparency logs, public records of issued certificates that let domain owners spot fraudulent ones, and to stricter audits.

PKI: Strengths, Weaknesses, Use Cases

Strengths

- Lets strangers establish trust at internet scale without sharing secrets first.
- One trust anchor can vouch for millions of identities.
- Certificates expire and can be revoked, so mistakes and thefts are recoverable.
- Open standards (X.509, TLS) work across every vendor.

Weaknesses

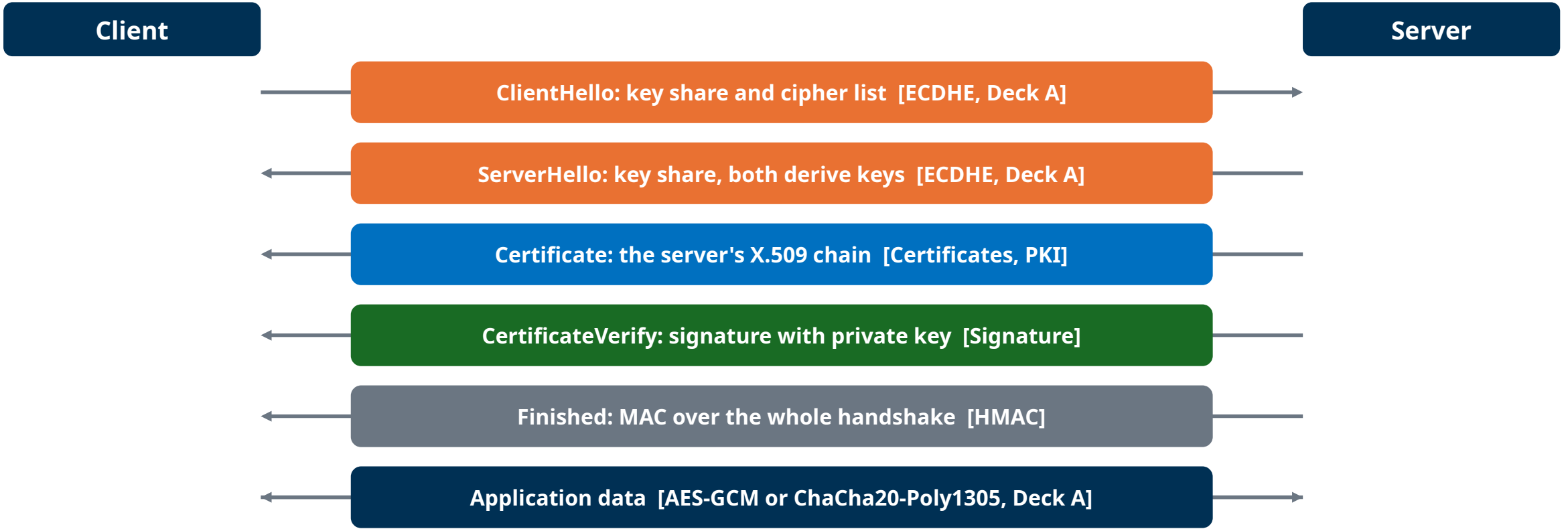
- Concentrates trust: a compromised CA can issue certificates for anyone (DigiNotar).
- Revocation is weak in practice: clients often skip or fail open on revocation checks.
- Operationally heavy: key protection, issuance, renewal, and expired certificates cause real outages.
- Users routinely click through browser warnings, defeating the model.

Use cases: HTTPS websites, SSL VPN and IPsec device authentication, 802.1X Wi-Fi with certificates, signed email (S/MIME), code signing, and device identity in enterprise networks.

Putting It Together

One handshake, every primitive

A TLS 1.3 Handshake, Primitive by Primitive



In TLS 1.3 everything after ServerHello, including the certificate, is already encrypted with keys derived from the key exchange.

Which Primitive Does What

Primitive	Provides	Needs	Example Algorithms
Symmetric encryption	Confidentiality of bulk data	A shared secret key	AES, ChaCha20
Key agreement	A shared secret over an open channel	Authentication of the peer	ECDHE, Diffie-Hellman
Hash function	A digest for integrity checks and signatures	Nothing secret	SHA-256, SHA-3
MAC / HMAC	Integrity and authentication among key holders	A shared secret key	HMAC-SHA256
Digital signature	Integrity, authentication, non-repudiation	A private key, and trust in the public key	RSA, ECDSA, Ed25519
Certificate and PKI	Trust that a public key belongs to a named identity	A trusted CA and a chain of trust	X.509, CA hierarchies

Cisco PKI Overview: Trustpoint and Enrolment

- A Cisco router that needs a certificate (for IPsec or SSL VPN authentication) first generates its own RSA key pair, then defines a trustpoint: a named object describing which CA to use, how to enrol, and which key to use.
- Authenticate: the router downloads the CA's own certificate and checks its fingerprint, ideally against a value obtained out of band. This is the step that establishes the root of trust.
- Enrol: the router sends a certificate request containing its public key and identity, and the CA returns the router's signed certificate.
- Once enrolled, VPN peers can authenticate with certificates instead of pre-shared keys, replacing the PSK mechanism from the Week 9 IPsec framework with the stronger, scalable option.

Cisco PKI Configuration Template

```
ip domain-name lab.local
crypto key generate rsa general-keys label RTR-KEYS modulus 2048
crypto pki trustpoint CORP-CA
 enrollment url http://192.168.1.50:80
 subject-name CN=R1.lab.local
 rsakeypair RTR-KEYS
 revocation-check none
crypto pki authenticate CORP-CA
crypto pki enroll CORP-CA
show crypto pki certificates
```

- Order matters: key pair first, then the trustpoint, then authenticate (fetch and verify the CA certificate), then enrol (get the router's own certificate).
- revocation-check none suits a lab only. In production, leave revocation checking on, so a revoked certificate is actually rejected.

Summary

- A hash gives a fixed-size fingerprint, but with no secret anyone can recompute it. A MAC adds a shared key for integrity and authentication. A digital signature uses a key pair to add non-repudiation.
- Signatures prove possession of a private key, not whose key it is. Certificates bind an identity to a public key, signed by a CA.
- PKI scales that trust through chains of trust from root stores, with expiry and revocation to recover from mistakes. It is only as strong as the CAs it trusts, as DigiNotar showed.
- A TLS handshake uses every primitive from both decks: key agreement, a certificate, a signature, a MAC, and finally symmetric encryption.

Coming Up

- Tutorial worksheets for both cryptography decks: study-notes questions for the mid-semester test and final exam.
- A hands-on practical to follow, built around hashing a file, tampering with it, and verifying a signature, so you see these primitives work rather than just read about them.

References

- Stallings, Cryptography and Network Security: Principles and Practice, 8th Ed., Pearson.
- Santos and Stuppi, CCNA Security 210-260 Official Cert Guide, Cisco Press.
- NIST FIPS 180-4, Secure Hash Standard, and FIPS 202, SHA-3 Standard.
- NIST FIPS 186-5, Digital Signature Standard.
- IETF RFC 2104, HMAC; RFC 5280, Internet X.509 PKI Certificate and CRL Profile; RFC 8446, TLS 1.3.